MUESTRA GRATUITA DEL LIBRO

Aprende C paso a paso

100 ejercicios resueltos

Rodrigo Iglesias

Una selección para conocer el recorrido del libro, leer dos ejercicios con su explicación completa y asomarte a uno de los proyectos finales.

Qué encontrarás en esta muestra

Índice completo - páginas 3-8 del libro.

Ejercicio 2 - suma de dos números, con código y explicación completos (páginas 21-22).

Ejercicio 44 - intercambio de valores mediante punteros, con código y explicación completos (páginas 118-119).

Fragmento avanzado - agenda de contactos: enunciado, propuesta de trabajo y cabecera inicial (páginas 285-286). La implementación y su explicación continúan en el libro y no se incluyen aquí.

Se conserva la paginación y la maquetación originales. El índice describe el libro completo, no solo esta selección. Las páginas 21 y 118 conservan breves continuaciones de los ejercicios anteriores que comparten página. Usa los marcadores del PDF para navegar.

Índice general

Introducción **9**

Cómo usar este libro **11**

Ruta de aprendizaje 13

Tu primer programa **15**

Código y material complementario **17**

Bloque 1: Introducción y fundamentos **19**

Ejercicio 1: Hola mundo 20

Ejercicio 2: Suma de dos números 21

Ejercicio 3: Entrada y salida 23

Ejercicio 4: Área de un círculo 24

Ejercicio 5: Conversión de segundos 25

Ejercicio 6: Código ASCII de un carácter 27

Ejercicio 7: Distancia entre dos puntos 28

Ejercicio 8: Promedio de tres números 30

Ejercicio 9: Intercambiar dos variables 31

Ejercicio 10: Volumen de una esfera 32

Bloque 2: Control de flujo **35**

Ejercicio 11: Número par o impar 36

Ejercicio 12: Máximo de dos números 37
Ejercicio 13: Positivo, negativo o cero 39
Ejercicio 14: Mayor de tres números 41
Ejercicio 15: Calculadora básica 42
Ejercicio 16: Factorial de un número (bucle for) 45
Ejercicio 17: Suma de los dígitos de un número (bucle while) 47
Ejercicio 18: Juego de adivinar un número 49
Ejercicio 19: Triángulo de asteriscos 51
Ejercicio 20: Números primos hasta N 53

Bloque 3: Funciones y modularidad **57**

Ejercicio 21: Función máximo de dos números 58
Ejercicio 22: Factorial con una función recursiva 60
Ejercicio 23: Función esPrimo 62
Ejercicio 24: Función MCD (Máximo Común Divisor) 65
Ejercicio 25: Potencia con exponente entero 67

Bloque 4: Operaciones a nivel de bit **71**

Ejercicio 26: Comprobar paridad con operadores de bit 72
Ejercicio 27: Contar bits a 1 en un número 74
Ejercicio 28: Verificar si un número es potencia de 2 76
Ejercicio 29: Intercambiar dos variables sin auxiliar (uso de XOR) 78
Ejercicio 30: Empaquetar dos bytes en un entero de 16 bits 80

Bloque 5: Vectores y cadenas de caracteres **83**

Ejercicio 31: Invertir el orden de un array 84
Ejercicio 32: Suma y promedio de un array 86

Ejercicio 33: Valor mínimo y máximo en un array 88
Ejercicio 34: Búsqueda lineal en un array 91
Ejercicio 35: Ordenar un array (método burbuja) 94
Ejercicio 36: Suma de dos matrices 96
Ejercicio 37: Multiplicación de matrices 99
Ejercicio 38: Longitud de una cadena sin strlen 101
Ejercicio 39: Contar vocales en una cadena 103
Ejercicio 40: Invertir una cadena de caracteres 106
Ejercicio 41: Comparar dos cadenas sin strcmp 108
Ejercicio 42: Búsqueda de subcadena sin strstr 111

Bloque 6: Punteros **115**

Ejercicio 43: Introducción a punteros 116
Ejercicio 44: Intercambiar dos valores usando punteros 118
Ejercicio 45: Recorrer array usando aritmética de punteros 120
Ejercicio 46: Copiar una cadena (implementación de strcpy) 122
Ejercicio 47: Puntero a función 124
Ejercicio 48: Buscar un carácter con punteros 126
Ejercicio 49: Inspeccionar bytes de un entero 128
Ejercicio 50: Punteros genéricos (`void *`) 130

Bloque 7: Memoria dinámica **133**

Ejercicio 51: Reservar y liberar un entero dinámicamente 134
Ejercicio 52: Array dinámico de N enteros 136
Ejercicio 53: Redimensionar un array dinámicamente (realloc) 138
Ejercicio 54: Concatenar dos cadenas con malloc 142

Ejercicio 55: Matriz dinámica 2D (reserva de doble puntero) 144
Ejercicio 56: Arreglo dinámico de cadenas 147

Bloque 8: Estructuras, uniones y enumeraciones **151**

Ejercicio 57: Definir y usar una estructura 152
Ejercicio 58: Arreglo de estructuras 154
Ejercicio 59: Unión para reinterpretar datos 157
Ejercicio 60: Enumeración de constantes simbólicas 159
Ejercicio 61: Reserva dinámica de una estructura 161
Ejercicio 62: Estructura con campos de bits 163

Bloque 9: Estructuras de datos **167**

Ejercicio 63: Lista enlazada simple 168
Ejercicio 64: Eliminar un nodo de la lista enlazada 171
Ejercicio 65: Invertir una lista enlazada 174
Ejercicio 66: Pila implementada con lista enlazada 177
Ejercicio 67: Cola implementada con lista enlazada 181
Ejercicio 68: Árbol binario de búsqueda 184
Ejercicio 69: Altura de un árbol binario 187
Ejercicio 70: Lista doblemente enlazada 191
Ejercicio 71: Ordenamiento rápido (Quicksort) recursivo 194
Ejercicio 72: Búsqueda binaria en array ordenado 197

Bloque 10: Archivos y manejo de ficheros **201**

Ejercicio 73: Escribir texto en un archivo 202
Ejercicio 74: Leer un archivo de texto 204
Ejercicio 75: Copiar un archivo de texto 206

Ejercicio 76: Contar líneas, palabras y bytes 209
Ejercicio 77: Escribir registros en un archivo binario 211
Ejercicio 78: Leer registros de archivo binario 214

Bloque 11: Compilación y preprocesador **217**

Ejercicio 79: Constantes de preprocesador 218
Ejercicio 80: Macros y evaluación de argumentos 220
Ejercicio 81: Compilación condicional con DEBUG 222
Ejercicio 82: Módulos y headers 224
Ejercicio 83: Uso de la macro assert 227

Bloque 12: Concurrencia **231**

Ejercicio 84: Primer hilo 232
Ejercicio 85: Suma en paralelo con 2 hilos 234
Ejercicio 86: Condición de carrera 237
Ejercicio 87: Uso de Mutex para sincronizar 239
Ejercicio 88: Productor-Consumidor con búfer 242

Bloque 13: Redes (sockets) **249**

Ejercicio 89: Cliente TCP sencillo 250
Ejercicio 90: Servidor TCP eco 254
Ejercicio 91: Comunicación UDP 258
Ejercicio 92: Servidor multicliente 262
Ejercicio 93: Cliente HTTP básico 266

Bloque 14: Seguridad y buenas prácticas **271**

Ejercicio 94: Eliminando buffer overflow 272

Ejercicio 95: Sustituir funciones inseguras 274
Ejercicio 96: Detectar overflow aritmético 277
Ejercicio 97: Uso de const en parámetros y variables 279
Ejercicio 98: Manejo adecuado de errores de funciones 282

Bloque 15: Proyectos finales **285**

Ejercicio 99: Agenda de contactos 285
Ejercicio 100: Calculadora en notación postfija 297

Seguir practicando **303**

Apéndice: herramientas y entorno **305**

Qué hace falta 305
Instalación 305
Comprobar la instalación 306
Configurar el editor 307
Estructura del proyecto 308
Un Makefile sencillo 309
Depurar 310
Sanitizers 310
Formato del código 311
Hábitos que merece la pena adquirir 311
Alternativa: Docker 312

Referencias **313**

Sobre el autor **315**

Enlaces 315

la salida. La secuencia `\n` representa un carácter de nueva línea; no imprime una barra y una letra. El punto y coma termina la instrucción de llamada.

`printf` escribe en la salida estándar, que normalmente está conectada a la terminal, pero también puede redirigirse a un archivo. El salto permite que la salida siguiente empiece en otra línea. La función devuelve la cantidad de caracteres escritos, o un valor negativo ante error; este ejemplo introductorio no comprueba ese retorno.

`return 0;` abandona `main` y comunica una terminación satisfactoria al entorno. Ese cero no es otro texto que aparezca en pantalla. El mensaje visible es el saludo; el estado del proceso es un dato separado que otros programas pueden consultar. En los ejercicios siguientes se usará uno para señalar los fallos en el entorno de trabajo del libro.

Al ejecutar el binario, la salida esperada es `Hola Mundo` en una línea. Editar el archivo fuente no modifica un ejecutable anterior: hay que compilar de nuevo para que un cambio del mensaje se refleje al ejecutarlo.

Ejercicio 2: Suma de dos números

Enunciado

Escribe un programa en C que pida dos números enteros, los sume y muestre el resultado. Para empezar, prueba con valores pequeños, como 5 y 7.

```c
#include <stdio.h>

int main(void) {
    int a, b, suma;

    printf("Introduce el primer número: ");
    scanf("%d", &a);

    printf("Introduce el segundo número: ");
    scanf("%d", &b);

    suma = a + b;

    printf("La suma de %d y %d es %d\n", a, b, suma);

    return 0;
}
```

Explicación del código

Hasta ahora el programa solo mostraba un mensaje fijo. Aquí su resultado depende de lo que escribas. Para guardar esos datos declaramos tres variables con `int a, b, suma;`: `a` y `b` contendrán los números de entrada, y `suma`, el resultado. `int` sirve para números enteros, como 5, 0 o -3.

Cada petición tiene dos instrucciones. `printf` muestra el mensaje y `scanf` lee el número que introduces. En `scanf("%d", &a)`, `%d` indica que se espera un entero y `&a` señala dónde guardarlo. De momento, recuerda esta pareja: para leer un `int`, usamos `%d` y escribimos `&` delante de la variable. Más adelante estudiarás con detalle qué representa esa dirección.

Las instrucciones se ejecutan en orden. Primero se guarda el primer número en `a`, luego el segundo en `b` y después se calcula `a + b`. En `suma = a + b`, el signo `=` asigna el resultado a la variable situada a su izquierda; no pregunta si los dos lados son iguales. Si introduces 5 y 7, `suma` pasa a valer 12. Las variables `a` y `b` siguen conservando sus valores.

El último `printf` tiene tres `%d`. Se sustituyen, en ese mismo orden, por `a`, `b` y `suma`. Con los datos del ejemplo verás `La suma de 5 y 7 es 12`. La secuencia `\n` termina la línea y `return 0;` finaliza el programa, como en el ejercicio anterior.

Puedes probar también con 5 y -2: el resultado será 3. Esta solución supone que introduces enteros pequeños. Como ampliación, cuando hayas aprendido `if`, podrás comprobar si la lectura se ha completado; en el ejercicio 96 estudiarás cómo detectar una suma que no cabe en el tipo. Esas comprobaciones hacen el programa más completo, pero no hacen falta para entender aquí la lectura, la suma y la salida.

Un paso más
Añade una impresión de &puntero y dibuja los dos objetos y la flecha entre ellos. Explica por qué no es lo mismo puntero que *puntero.

Ejercicio 44: Intercambiar dos valores usando punteros

Enunciado

Escribe una función `swap(int *a, int *b)` que intercambie los valores de dos variables utilizando punteros para simular el paso por referencia. Prueba su funcionamiento en `main`.

```c
#include <stdio.h>

// Función que intercambia los valores de dos enteros usando punteros
void swap(int *a, int *b) {
    int temp = *a;  // Guardar el valor apuntado por 'a' en 'temp'
    *a = *b;        // Asignar el valor apuntado por 'b' a la dirección apuntada por 'a'
    *b = temp;      // Asignar el valor temporal a la dirección apuntada por 'b'
}

int main(void) {
    int x = 5, y = 10;

    printf("Antes del intercambio:\n");
    printf("x = %d, y = %d\n", x, y);

    swap(&x, &y); // Pasar las direcciones de x e y para modificar sus valores

    printf("Después del intercambio:\n");
    printf("x = %d, y = %d\n", x, y);

    return 0;
}
```

Explicación del código

La función debe modificar los dos enteros que existen en `main`. Por eso `swap(&x, &y)` pasa sus direcciones: el parámetro `a` recibe una copia de `&x` y `b`, una copia de `&y`. C sigue pasando argumentos por valor; compartir el acceso a los objetos permite el efecto que suele llamarse paso por referencia simulado.

Los parámetros tienen tipo `int *`. La expresión `*a` designa el entero apuntado por `a`, y `*b` designa el apuntado por `b`. Las asignaciones de la función escriben en esos enteros, sin cambiar las direcciones guardadas en los parámetros. Reasignar solamente `a` o `b` movería el acceso local y no intercambiaría los valores del llamador.

El temporal es necesario para conservar un valor antes de sobrescribirlo. En `int temp = *a`, se copia el valor inicial de `x`, que es `5`. A continuación, `*a = *b` escribe el `10` de `y` en `x`: temporalmente ambas variables valen diez, pero el cinco sigue guardado en `temp`. Finalmente, `*b = temp` escribe ese cinco en `y`. Las tres instrucciones dejan `x = 10` e `y = 5`.

Si se eliminara el temporal y se efectuaran `*a = *b` y después `*b = *a`, la segunda operación leería el valor ya sobrescrito: ambos enteros acabarían con el valor original de `y`. La variable auxiliar evita esa pérdida sin recurrir a sumas o productos que pudieran desbordarse. El intercambio funciona igualmente con negativos o valores extremos representables.

La función devuelve `void` porque el resultado está en los objetos modificados y no necesita un valor de retorno. Su contrato exige dos direcciones válidas de enteros vivos, inicializados y modificables. No contiene comprobaciones para punteros nulos ni puede detectar de forma general si una dirección arbitraria apunta a memoria válida.

También funciona si ambas direcciones identifican el mismo entero, como en `swap(&x, &x)`: el temporal conserva su valor y las asignaciones lo dejan igual. Que dos parámetros puedan referirse al mismo objeto se conoce como alias; no debe suponerse siempre que apuntan a variables diferentes.

Las impresiones antes y después muestran el efecto sobre `x` e `y`, sin depender de direcciones concretas. Pasar `x` e `y` directamente a esta interfaz no produciría dos copias utilizables como punteros: incumpliría sus tipos y requeriría un diagnóstico. Una función distinta declarada con parámetros `int` trabajaría con copias y no podría efectuar este intercambio en el llamador.

Bloque 15: Proyectos finales

Los dos proyectos finales reúnen funciones, cadenas, estructuras de datos, memoria y archivos. Cada uno tiene un alcance acotado que podrás ampliar cuando entiendas cómo organiza sus operaciones y cómo trata los errores.

La **agenda de contactos** guarda nombres, teléfonos y correos electrónicos. Un menú permite añadir, buscar y eliminar contactos, y guardar o recuperar los datos desde un archivo. Estudiarás tanto la lista en memoria como las condiciones de lectura y escritura.

La **calculadora en notación postfija** (RPN) evalúa expresiones postfijas con una pila. Tiene que distinguir números de operadores, comprobar que hay operandos suficientes y gestionar los errores del cálculo. Ninguno de los dos proyectos usa hilos ni redes: cada uno emplea solo las herramientas que su problema necesita.

Índice general

Ejercicio 99: Agenda de contactos 285
Ejercicio 100: Calculadora en notación postfija 297

Ejercicio 99: Agenda de contactos

Enunciado

Construye una agenda con alta, búsqueda, edición de teléfono y email, borrado, listado y persistencia en un archivo de texto. Usa una lista enlazada y separa las operaciones en módulos.

Cada campo admite hasta 99 bytes. El nombre es obligatorio; teléfono y email pueden estar vacíos. El separador | no se admite dentro de los campos. La búsqueda distingue mayúsculas y minúsculas y actúa sobre la primera coincidencia. La carga sustituye la agenda existente solo si consigue leer todo el archivo; cargar dos veces no duplica los

datos. Guardar sobrescribe agenda.txt y, si falla una escritura, puede dejar un archivo incompleto.

Antes de ver la solución

Divide el trabajo en cuatro etapas. Primero crea y libera una lista en memoria. Después incorpora búsqueda, edición y borrado. A continuación añade el menú. Por último guarda, reinicia y carga para comprobar que no pierdes datos.

La estructura de archivos es main.c, contacto.c, contacto.h, persistencia.c y persistencia.h. Copia cada bloque en el archivo que indica su encabezado.

`contacto.h`

```c
#ifndef CONTACTO_H
#define CONTACTO_H
#define TAM 100

typedef struct {
    char nombre[TAM];
    char telefono[TAM];
    char email[TAM];
} Contacto;

typedef struct Nodo {
    Contacto contacto;
    struct Nodo *siguiente;
} Nodo;

void agregarContacto(Nodo **inicio);
void listarContactos(const Nodo *inicio);
void buscarContacto(const Nodo *inicio);
void eliminarContacto(Nodo **inicio);
void editarContacto(Nodo *inicio);
void liberarLista(Nodo *inicio);
#endif
```

SIGUE APRENDIENDO C

De los primeros cálculos a proyectos completos.

El libro reúne 100 ejercicios en 15 bloques: fundamentos, control de flujo, funciones, bits, arrays y cadenas, punteros, memoria, estructuras, archivos, concurrencia, redes y proyectos finales.

En esta muestra has leído dos ejercicios completos. El proyecto de agenda termina aquí en su cabecera inicial: faltan la implementación de los módulos y su explicación. No es un programa completo para compilar.

Ver el libro completo en Amazon

Código complementario
github.com/RigleGit/c100-ejercicios
Ejercicios, documentación y entorno de trabajo sin registro.

Continúa practicando en la web
c.rigle.dev

Aprende C paso a paso: 100 ejercicios resueltos
Rodrigo Iglesias